

Part 1 Research + Reverse Engineering

Project: College Decision Platform

Date: 2026-06-08

Objective

This document deconstructs the college discovery ecosystem across Collegedunia, Careers360, Shiksha, GetMyUni, and Zollege. The goal is to understand the product surface, the data behind each page, and the backend model needed to rebuild a cleaner decision-first MVP.

The product direction is not to clone any competitor. The opportunity is to reduce clutter, make comparison easier, and help students move from discovery to shortlist faster.

1. Page Architecture Breakdown

Major page types found across the ecosystem:

1. Homepage
2. College listing pages
3. College detail pages
4. Course pages
5. Exam pages
6. Tools: predictors, compare, finder
7. Rankings and collections
8. Q&A and community
9. Articles and content
10. User features: login, saved colleges, profile, alerts

2. Feature + Data Deconstruction

Page: Homepage

Objective: Help users start discovery through search, category navigation, and prominent entry points such as colleges, exams, courses, rankings, and tools.

Features:

- Global search for colleges, courses, exams, and articles
- Category navigation by stream such as engineering, MBA, medical, law, arts, design
- Popular colleges, popular exams, and featured rankings
- Tool entry points such as predictor, compare, finder, and counselling
- Login and user account entry
- SEO content blocks and high-intent links

Data Points:

- Search index terms: college name, course name, exam name, city, state, stream
- Featured college fields: name, city, state, rating, fees, ranking, placement summary
- Featured course fields: course name, degree, stream, duration, average fees

- Featured exam fields: exam name, application date, exam date, mode, accepted colleges
- User state: logged in, saved count, recent searches

Backend Thinking:

- Search service should query colleges, courses, and exams
- Homepage data should be cacheable and read-optimized
- Featured blocks should be CMS/config driven, not hardcoded in UI

Page: College Listing Page

Objective: Let students discover and shortlist colleges using search, filters, sorting, pagination, and comparison.

Features:

- Search by college name, city, state, course, or exam
- Filters for location, course, fees, exams accepted, rating, ranking, ownership, and placement
- Sorting by rating, fees, ranking, popularity, placement, or relevance
- Paginated college cards
- Save/bookmark action
- Compare selection
- Empty state when no results match

Data Points:

- College card: id, name, city, state, ownership, rating, ranking, fees range, placement percentage, average package, accepted exams, course tags, review count
- Filter metadata: cities, states, courses, exams, fee ranges, rating ranges
- Pagination: page, limit, total, total pages
- User-specific fields: saved status, compare selected status

Backend Thinking:

- Listing API must support indexed filtering and pagination
- Search should cover college name, location, and related course names
- Filter options should come from API metadata, not frontend constants
- Null placement/package data must be returned safely and displayed as unavailable

Page: College Detail Page

Objective: Give a structured decision view for one college so the student can judge fit without opening many tabs.

Features:

- Top summary with rating, fees, placement, average package, ranking
- Location and overview
- Courses offered
- Fees and eligibility
- Admission process
- Exams accepted

- Placement snapshot
- Reviews and student sentiment
- Facilities and campus highlights
- Similar colleges
- Save and compare actions

Data Points:

- College: id, name, description, city, state, address, ownership, established year, accreditation, affiliation
- Metrics: rating, ranking, fees range, placement percentage, average package, highest package
- Courses: course id, name, stream, degree, duration, annual fee, eligibility, seats
- Exams: exam id, name, cutoff, application status, accepted course
- Reviews: rating, title, text, reviewer profile, created date, sentiment tags
- Facilities: hostel, library, labs, sports, transport, medical, wifi
- Decision tags: affordable, high placement, strong rating, premium fees, data incomplete

Backend Thinking:

- Detail endpoint should aggregate college, courses, exams, and decision tags
- Detail view should tolerate missing placement or package fields
- Courses and exams should be modeled as relationships, not text blobs

Page: Course Pages

Objective: Help users understand a course category and find colleges offering it.

Features:

- Course overview
- Eligibility and duration
- Average fees
- Career outcomes
- Top colleges offering course
- Exams required
- Related courses

Data Points:

- Course: id, name, stream, degree, duration, description
- Course-college relationship: college id, course id, annual fee, seats, eligibility
- Outcome fields: common roles, salary range, placement relevance
- Exams: required or accepted exams for admission

Backend Thinking:

- Course pages require many-to-many relationship between colleges and courses
- Course filters should reuse listing API logic

Page: Exam Pages

Objective: Help students understand entrance exams and discover colleges accepting them.

Features:

- Exam overview
- Important dates
- Eligibility
- Syllabus and pattern
- Application process
- Results and counselling
- College predictor entry
- Colleges accepting exam

Data Points:

- Exam: id, name, stream, level, mode, conducting body
- Dates: registration start/end, admit card, exam date, result date, counselling date
- Exam-college relation: accepted colleges, accepted courses, cutoff ranges
- Content: syllabus sections, pattern, marking scheme, article links

Backend Thinking:

- Exam data should be normalized because it powers exam pages, listing filters, and predictors
- Date fields should support nulls and stale-date warnings

Page: Tools: Predictor, Compare, Finder

Objective: Convert raw data into decision support.

Features:

- College predictor using rank, exam, category, state, quota, branch preference
- Compare 2 to 3 colleges side by side
- College finder using preferences such as budget, course, city, rating, placement
- Recommendation results with reasoning

Data Points:

- Predictor input: exam id, rank, score, category, domicile, gender, branch, round
- Predictor output: college id, course id, chance level, previous cutoff, counselling note
- Compare fields: fees, rating, placement, average package, ranking, courses, exams
- Finder preferences: location, fee cap, course, minimum rating, placement priority

Backend Thinking:

- Compare endpoint should validate 2 to 3 ids
- Finder can start rule-based and later become recommendation-driven
- Predictors require historical cutoff data and should be separated from core college listing

Page: Rankings and Collections

Objective: Organize colleges into intent-based discovery groups such as top engineering colleges, best MBA colleges in Hyderabad, or affordable private colleges.

Features:

- SEO-friendly collection pages
- Ranking tables
- Filters within rankings
- College cards or table rows
- Editorial notes and methodology

Data Points:

- Collection: id, slug, title, stream, city/state, ranking source, description
- Ranking: college id, rank, source, year, category
- College summary: name, location, fees, rating, placement, ownership

Backend Thinking:

- Collections can be generated from filters or stored as curated pages
- Ranking source and year must be stored to avoid misleading users

Page: Q&A / Community

Objective: Let students ask admission, course, exam, and college-specific questions.

Features:

- Ask question
- Answer threads
- College/course/exam tagging
- Upvotes or helpful marks
- Moderation
- Related questions on detail pages

Data Points:

- Question: id, title, body, user id, tags, college id, course id, exam id, created date
- Answer: id, question id, user id, body, helpful count, verified flag
- User: name, role, credibility score

Backend Thinking:

- Community content is user-generated and requires moderation fields
- Tags connect questions back to discovery pages

Page: Articles / Content

Objective: Capture informational searches and guide students into colleges, exams, rankings, and tools.

Features:

- News and guides
- Admission updates
- Exam result updates
- College/course explainers

- Related college and exam widgets

Data Points:

- Article: id, title, slug, body, category, author, published date, updated date
- Relations: college ids, course ids, exam ids, city/state tags
- SEO fields: meta title, meta description, canonical URL

Backend Thinking:

- Content should be a separate CMS-like model
- Related entities should be stored as references so articles can drive discovery

Page: User Features

Objective: Preserve user intent across sessions and support shortlisting.

Features:

- Login/register
- Saved colleges
- Saved searches
- Compare history
- Alerts for exams/admissions
- Profile preferences

Data Points:

- User: id, name, email, created date
- Saved college: user id, college id, created date
- Preference: user id, course, city, max fees, minimum rating
- Alert: user id, entity type, entity id, event type

Backend Thinking:

- Saved colleges require user-specific queries
- MVP can use lightweight email login, but production should use secure auth

3. Data Model Thinking

Core Entities

College:

- id
- name
- slug
- city
- state
- address
- ownership
- established_year
- accreditation
- affiliation

- fees_range
- rating
- placement_percentage
- avg_package
- highest_package
- ranking
- description
- created_at
- updated_at

Course:

- id
- name
- slug
- stream
- degree
- duration
- description

CollegeCourse:

- id
- college_id
- course_id
- annual_fee
- seats
- eligibility
- placement_percentage

Exam:

- id
- name
- slug
- stream
- level
- mode
- conducting_body
- application_start
- application_end
- exam_date
- result_date

CollegeExam:

- id
- college_id
- exam_id
- course_id
- cutoff_min
- cutoff_max

- admission_round

User:

- id
- name
- email
- created_at
- updated_at

SavedCollege:

- id
- user_id
- college_id
- created_at

Review:

- id
- user_id
- college_id
- course_id
- rating
- title
- body
- placement_rating
- faculty_rating
- campus_rating
- verified
- created_at

Question:

- id
- user_id
- college_id
- course_id
- exam_id
- title
- body
- created_at

Answer:

- id
- question_id
- user_id
- body
- helpful_count
- verified
- created_at

Ranking:

- id
- college_id
- source
- category
- year
- rank

Relationships

- College has many Courses through CollegeCourse
- College accepts many Exams through CollegeExam
- Exam maps to many Colleges and Courses
- User saves many Colleges through SavedCollege
- User writes many Reviews
- College has many Reviews
- College, Course, and Exam can be attached to Questions
- Ranking belongs to College and is scoped by source/category/year

Read-Heavy Optimization

- Index colleges by city, state, rating, fees_range, and ranking
- Index many-to-many tables by college_id and course_id
- Use pagination with LIMIT/OFFSET for MVP
- Add full-text search or trigram search when data grows
- Cache metadata endpoints such as filter options
- Keep listing payload compact and load full details only on detail page

4. Competitive Analysis

| Platform | Strength | Weakness | Missing |

|---|---|---|---|

| Collegedunia | Large SEO footprint, broad college/course/exam coverage, strong lead-generation flows | Pages can feel cluttered, heavy content density, decision path is not always clear | Cleaner comparison, transparent confidence, fewer distractions, faster shortlist workflow |

| Careers360 | Strong tools ecosystem, predictors, rankings, exam coverage, counselling-oriented flows | Tool flows can feel gated or complex, many content paths compete for attention | Lightweight decision dashboard, simple compare-first UX, clearer explanation of recommendation logic |

| Shiksha | Deep course and review coverage, strong college/course detail pages, useful filters | Review trust can be hard to judge, dense pages, sponsored/lead flows may distract |

| Verified review signals, cleaner metric hierarchy, stronger save-to-decision workflow |

| GetMyUni | Admission-focused content, college discovery, counselling funnel, simplified summaries | Less structured decision tooling, content sometimes feels lead-oriented |

Stronger comparison, richer structured data, transparent fit scoring |

| Zollege | Simple college pages, SEO collections, accessible listing style | Shallower tooling and decision support compared with larger competitors | Better backend-driven

filters, compare, saved lists, recommendation logic |

Platform-Specific Observations

Collegedunia does best at breadth and SEO. It covers colleges, courses, exams, reviews, rankings, and admission content at scale. Its weakness is information overload: a student often has to interpret many unrelated blocks before making a decision.

Careers360 does best at tools such as college predictors, rank predictors, comparisons, and counselling journeys. Its weakness is complexity: tool flows require many inputs and can feel heavier than needed for early-stage exploration.

Shiksha does best at structured course/college information and reviews. Its weakness is trust and clarity: students may struggle to understand which reviews are reliable and which metrics matter most.

GetMyUni does best at admission-oriented discovery and guidance. Its weakness is weaker decision tooling: compare and shortlist flows are not as central as they should be.

Zollege does best at simpler SEO-driven discovery pages. Its weakness is data depth and product tooling: it is useful for browsing but less strong for actual decision-making.

Market Gaps

1. Weak comparison UX: Most platforms provide comparison, but it is usually table-heavy and not decision-ranked.
2. Low personalization: Discovery is mostly filter-based, not preference-based.
3. Poor decision support: Users see many facts but few clear conclusions.
4. Cluttered pages: Lead forms, ads, articles, rankings, and widgets often compete with the student decision.
5. Missing data confidence: Platforms do not clearly show whether placement/package data is missing, estimated, stale, or verified.
6. Shortlisting is underpowered: Saved colleges often behave like bookmarks, not a decision workspace.
7. Performance friction: Large pages can feel slow because they combine content, ads, tools, and widgets.

Product Direction for the MVP

The MVP should focus on:

- Fast college discovery
- API-driven filters for city and course
- Clean college cards with decision metrics
- Detail pages that prioritize rating, fees, placement, and package
- Compare flow for 2 to 3 colleges
- User-specific saved colleges
- Missing-data fallbacks
- Consistent backend responses

This creates a narrower but clearer product than the incumbents. The build prioritizes decision speed over content volume.

Sources Reviewed

- Collegedunia: <https://collegedunia.com/>
- Careers360: <https://www.careers360.com/>
- Shiksha: <https://www.shiksha.com/>
- GetMyUni: <https://www.getmyuni.com/>
- Zollege: <https://zollege.in/>
- Careers360 College Predictor examples:
<https://engineering.careers360.com/jee-main-college-predictor>
- Supplied assignment brief and product requirements